

Aufgabenspezifikation

Tutorial | Bio-inspirierte Optimierungsalgorithmen • Schwarmintelligenz

Modul	Angewandte Informatik
Thema	Bio-inspirierte Optimierungsalgorithmen
Sprache	Java SE Development Kit 25.0.3 (nur LTS)
JDK-Download	https://www.oracle.com/de/java/technologies/downloads/###java25
Abgabe MOS	08. Mai 2026 EOB
Abgabe MGH	13. Mai 2026 EOB
Abgabeformat	[matrikelnummer].zip mit vollständigem src -Verzeichnis

1. Lernziele

Diese Aufgabe fördert den kompetenten Umgang mit bio-inspirierten Algorithmen, modernen KI-assistierte Entwicklungswerkzeugen sowie den Prinzipien professioneller Softwareentwicklung. Nach Abschluss der Aufgabe sollen Sie in der Lage sein:

- eine gegebene Implementierung eines Optimierungsalgorithmus kritisch zu analysieren und gezielt zu verbessern,
- KI-Tools methodisch und reflektiert als Hilfsmittel im Softwareentwicklungsprozess einzusetzen,
- Prinzipien des Clean Coding und SOLID-Designs in bestehendem Code zu identifizieren und anzuwenden,
- algorithmisches Verhalten (Agentendynamik, Konvergenz, Gedächtnisstrukturen) strukturiert zu beobachten und zu protokollieren,
- professionelle JavaDoc-Dokumentation zu erstellen.

2. Algorithmen

Für dieses Tutorial stehen die folgenden 16 Algorithmen zur Verfügung. Die individuelle Zuweisung eines Algorithmus erfolgt durch die Lehrperson; kein Algorithmus wird doppelt vergeben.

Nr.	Algorithmus	Kürzel
1	Artificial Bee Colony	ABC
2	Artificial Hummingbird Algorithm	AHA
3	Bat Algorithm	BA

Nr.	Algorithmus	Kürzel
4	Chameleon Swarm Algorithm	CSA
5	Chemical Reaction Optimization	CRO
6	Dolphin Swarm Algorithm	DSA
7	Elephant Herding Optimization	EHO
8	Fireworks Algorithm	FWA
9	Grasshopper Optimization Algorithm	GOA
10	Grey Wolf Optimization	GWO
11	Harris Hawks Optimizer	HHO
12	Lion Optimization Algorithm	LOA
13	Tuna Swarm Optimization	TSO
14	Whale Optimization Algorithm	WOA
15	Wild Horse Optimizer	WHO
16	Zebra Optimization Algorithm	ZOA

3. Aufgabenstellung

Aufgabe 1 • Analyse und Korrektur der Implementierung

Kontext: Gegeben ist eine Basis-Implementierung des Ihnen zugewiesenen Algorithmus in Java.

Vorgehen:

1. Setzen Sie ein kostenfreies KI-Tool Ihrer Wahl ein (Empfehlungen siehe Abschnitt 4).
2. Analysieren Sie die Basis-Implementierung **systematisch und sorgfältig** auf korrektes algorithmisches Verhalten.
 - Orientieren Sie sich dabei an der wissenschaftlichen Originalpublikation bzw. anerkannten Referenzquellen zum Algorithmus.
 - Prüfen Sie insbesondere: Initialisierung der Agenten/Population, korrekte Umsetzung der algorithmischen Kernoperatoren (z. B. Phasen, Bewegungsregeln, Selektionsmechanismen), Abbruchbedingungen sowie die Behandlung von Randfällen.
3. Dokumentieren Sie alle identifizierten Inkonsistenzen und Fehler nachvollziehbar.
4. Beheben Sie alle identifizierten Inkonsistenzen im Quellcode.

Abzugebende Artefakte:

- Korrigierter Java-Quellcode
- Kurze schriftliche Zusammenfassung der identifizierten Probleme und vorgenommenen Korrekturen (in `CHANGES.md` oder als JavaDoc auf Klassenebene)

Aufgabe 2 • Analyse und Verbesserung des Testszenarios

Kontext: Zur Implementierung liegt ein Basis-Testszenario vor.

Vorgehen:

1. Analysieren Sie das bestehende Testszenario mithilfe eines kostenfreien KI-Tools auf folgende Aspekte:
 - **Korrektheit:** Testen die vorhandenen Tests das tatsächlich beabsichtigte Verhalten?
 - **Vollständigkeit:** Werden alle wesentlichen Aspekte des Algorithmus (Initialisierung, Iteration, Konvergenzbedingung, Randfälle) abgedeckt?
 - **Konsistenz:** Sind Testbezeichnungen, Assertions und Testdaten widerspruchsfrei und aussagekräftig?
2. Dokumentieren Sie alle festgestellten Mängel.
3. Beheben Sie alle identifizierten Mängel und ergänzen Sie fehlende Testfälle.

Abzugebende Artefakte:

Überarbeitetes und erweitertes Testszenario (JUnit 5 und assertJ)

Aufgabe 3 • Clean Coding und SOLID-Prinzipien

Kontext: Softwarequalität ist ein zentrales Merkmal professioneller Softwareentwicklung.

Vorgehen:

1. Analysieren Sie die bereits korrigierte Implementierung mithilfe eines kostenfreien KI-Tools auf die Einhaltung folgender Prinzipien:

Prinzip	Beschreibung (Kurzfassung)
Single Responsibility (SRP)	Jede Klasse/Methode hat genau eine klar definierte Verantwortung
Open/Closed (OCP)	Erweiterbar ohne Modifikation bestehenden Codes
Liskov Substitution (LSP)	Untertypen verhalten sich substituierbar wie ihre Basistypen
Interface Segregation (ISP)	Keine Abhängigkeit von ungenutzten Schnittstellen
Dependency Inversion (DIP)	Abhängigkeiten zeigen auf Abstraktionen, nicht auf Konkretionen
Clean Coding	Sprechende Namen, kurze Methoden, kein toter Code, keine Magic Numbers, DRY, KISS

2. Dokumentieren Sie alle festgestellten Verstöße mit konkretem Bezug zur betroffenen Klasse bzw. Methode.
3. Refaktorisieren Sie den Code, um die Einhaltung der Prinzipien weitestgehend sicherzustellen. Begründen Sie alle vorgenommenen Refaktorisierungsmaßnahmen kurz in `CHANGES.md`.

Abzugebende Artefakte:

- Refaktorisierter Quellcode
- Begründung der Refaktorisierungsmaßnahmen in `CHANGES.md`

Aufgabe 4 • Logging: Agentendynamik und Konvergenz an der Ackley-Funktion

Kontext: Das Verständnis des algorithmischen Verhaltens • insbesondere Agentendynamik, Gedächtnisstrukturen und Konvergenz • ist ein zentrales Lernziel dieses Tutorials.

Testfunktion: Die **Ackley-Funktion** (2D) dient als einheitliches Bewertungsszenario:

$$f(x) = -a * \exp(-b * \sqrt{1/n * \sum(x_i^2)}) - \exp(1/n * \sum(\cos(c * x_i))) + a + e$$

Standardparameter: $a=20$, $b=0.2$, $c=2*\pi$

Suchraum: $[-32.768, 32.768]^n$

Globales Minimum: $f(0, \dots, 0) = 0$

Vorgehen:

1. Entwickeln Sie mithilfe eines kostenfreien KI-Tools ein Logging-Konzept, das folgende Aspekte **pro Iteration** in einem strukturierten Logfile (`algorithm_run.log`) nachvollziehbar protokolliert:

Aspekt	Zu protokollierende Informationen
Agenten-Interaktion	Positionsupdates je Agent (vorher / nachher), ggf. Kommunikation und Kooperation zwischen Agenten
Lokales Gedächtnis	Beste bisher gefundene Position und Fitnesswert je Agent (Personal Best)
Globales Gedächtnis	Beste global bekannte Position und Fitnesswert der gesamten Population (Global Best)
Konvergenz	Durchschnittliche Fitness der Population, Standardabweichung, Abstand zum globalen Optimum

2. Implementieren Sie das Konzept direkt in die bestehende Java-Implementierung.
3. Das Logfile soll maschinell auswertbar (z. B. CSV oder strukturiertes Textformat) **und** für Menschen gut lesbar sein.
4. Stellen Sie sicher, dass das Logging deaktivierbar ist (z. B. über eine Konfigurationsklasse oder ein Flag), sodass es den produktiven Betrieb nicht beeinträchtigt.

Mindestinhalt einer Logzeile (Beispielformat, CSV):

```
iteration;agentId;positionBefore;positionAfter;personalBest;personalBestFitness;globalBest;globalBestFitness;popAvgFitness;popStdDev;distToOptimum
```

Abzugebende Artefakte:

- Erweiterter Quellcode mit integriertem Logging
- Beispiel-Logfile einer vollständigen Ausführung auf der Ackley-Funktion (2D)

Aufgabe 5 • JavaDoc-Kommentierung

Kontext: Professionelle Dokumentation ist Bestandteil jeder qualitativ hochwertigen Softwareentwicklung.

Vorgehen:

1. Identifizieren Sie alle **zentralen, nicht-trivialen Methoden** in Implementierung und Tests.

Faustregel: Methoden mit nicht-offensichtlicher Logik, algorithmischen Kernoperatoren, komplexen Parametern oder Seiteneffekten sind zu kommentieren. Triviale Getter/Setter sowie selbsterklärende Hilfsmethoden können ausgelassen werden.

2. Generieren Sie mithilfe eines KI-Tools prägnante und aussagekräftige JavaDoc-Kommentare für diese Methoden.
3. Stellen Sie sicher, dass die Kommentare folgende Elemente enthalten:

Tag	Inhalt
<i>(Beschreibung)</i>	Was tut die Methode? Warum existiert sie im algorithmischen Kontext?
<code>@param</code>	Jeder Parameter mit Typ und inhaltlicher Erläuterung
<code>@return</code>	Erläuterung des Rückgabewerts (sofern vorhanden)
<code>@throws</code>	Alle deklarierten oder möglichen Ausnahmen
<i>(optional)</i> <code>@see</code>	Verweis auf Formel oder Abschnitt aus der Originalpublikation

Qualitätskriterium: Kommentare dürfen nicht lediglich den Methodennamen paraphrasieren. Sie müssen einen echten Mehrwert für das Verständnis des Codes liefern.

Abzugebende Artefakte:

- Vollständig kommentierter Quellcode (Implementierung und Tests)

4. Einsatz von KI-Tools

Der Einsatz kostenfreier KI-Tools ist für alle Aufgaben ausdrücklich vorgesehen und erwünscht. Bitte beachten Sie dabei folgende Hinweise:

- **Transparenz:** Geben Sie in `CHANGES.md` an, welches KI-Tool Sie für welche Aufgabe genutzt haben.
- **Kritische Reflexion:** KI-generierte Ergebnisse sind eigenverantwortlich zu prüfen und ggf. zu korrigieren. Fehler im abgegebenen Code, die auf unkritisch übernommene KI-Ausgaben zurückzuführen sind, gelten als Qualitätsmängel.
- **Eigenleistung:** Das Verstehen, Beurteilen und Integrieren der KI-Ausgaben ist Kernbestandteil der geforderten Eigenleistung.

Empfohlene kostenfreie Tools (ohne Präferenz der Lehrperson):

Tool	URL
Anthropic Claude	https://claude.ai
ChatGPT (Free Tier)	https://chatgpt.com
DeepSeek	https://chat.deepseek.com
Google Gemini	https://gemini.google.com
GitHub Copilot (Student)	https://github.com/features/copilot
Microsoft Copilot	https://copilot.microsoft.com
Mistral Le Chat	https://chat.mistral.ai

5. Qualitätskriterien

Eine hohe Qualität der Abgabe ist im Hinblick auf einen optimalen Lernerfolg ausdrücklich sicherzustellen. Die folgenden Kriterien dienen als Orientierung für eine professionelle und vollständige Bearbeitung:

Kriterium	Beschreibung
Korrektheit	Der Algorithmus verhält sich gemäß seiner Spezifikation; alle Tests bestehen
Testvollständigkeit	Wesentliche Aspekte des Algorithmus sind durch aussagekräftige Tests abgedeckt
Clean Code	Code ist lesbar, wartbar und frei von redundantem oder totem Code
SOLID	Prinzipien sind weitestgehend eingehalten; Abweichungen sind in <code>CHANGES.md</code> begründet
Logging	Das Logfile ist vollständig, strukturiert und nachvollziehbar
JavaDoc	Kommentare sind präzise, vollständig und inhaltlich korrekt
KI-Nutzung	KI-Tools wurden reflektiert eingesetzt; Ergebnisse wurden kritisch geprüft
Nachvollziehbarkeit	Alle Änderungen gegenüber der Basis-Implementierung sind dokumentiert

6. Abgabestruktur

Die Abgabe erfolgt als ZIP-Datei mit dem Dateinamen `[matrikelnummer].zip`. Das Archiv muss folgende Struktur aufweisen:

```
[matrikelnummer].zip
├─ src/
│   └─ main/
│       └─ java/
│           └─ [package]/
│               └─ [AlgorithmName].java      ← Hauptimplementierung
```

```

|       |─ AckleyFunction.java      ← Testfunktion (Ackley)
|       |─ AlgorithmLogger.java    ← Logging-Komponente
|       └─ ...                     ← weitere Klassen
|─ test/
|   └─ java/
|       └─ [package]/
|           └─ [AlgorithmName]Test.java ← JUnit-5-Tests
|─ resources/
|   └─ algorithm_run.log           ← Beispiel-Logfile
└─ CHANGES.md                    ← Änderungsprotokoll + KI-Tool-Angaben

```

Hinweis: Build-Artefakte (`.class`-Dateien, `/target`- oder `/out`-Verzeichnis) sind **nicht** in das Archiv aufzunehmen.

7. Abgabefristen

Kurs	Abgabefrist	Hinweis
MOS	08. Mai 2026	Abgabe über die Lernplattform
MGH	13. Mai 2026	Abgabe über die Lernplattform